Modelo de Predição

Nádia Cristina Pinheiro Rodrigues

Baseado na disciplina “Modelos de Predição em Saúde”
ministrada pelo Professor Alexandre Chiavegatto Filho (FSP/USP)

Qual o melhor algoritmo?

Para diferentes conjuntos de dados, não existe apenas 1 algoritmo que sempre tenha a melhor performance.

Por isso, é necessário testar qual terá a melhor performance em um conjunto específico de dados.

Por outro lado, alguns algoritmos têm se mostrado mais promissores nos últimos tempos - gradient boosting, random forest, TabPFN.

Estimando a performance dos algoritmos

Configurações diferentes do mesmo algoritmo resultam em performances diferentes

Por isso a necessidade de encontrar a melhor configuração do algoritmo ainda na fase de treino

Por que avaliar o desempenho de algoritmos?

✓Comparar e selecionar o melhor algoritmo de ML

✓Estimar a capacidade de generalização (futuro)

Função alvo, modelo e algoritmo

Função alvo

Representa a **relação real** entre os preditores (x) e o desfecho (y).

Exemplo: se queremos prever a cura do paciente de TB, a função alvo é a relação **real** entre características demográficas, clínicas, comportamentais, laboratoriais, entre outras, que levam a prever a cura da doença.

$$y = f(x)$$

Modelo – aproximação da função alvo

É uma função desenvolvida pelo pesquisador que busca se aproximar da função-alvo.

Exemplo: podemos usar um modelo para estimar a probabilidade de cura da TB com base em fatores demográficos, clínicos, comportamentais e laboratoriais.

O modelo tentará reproduzir a relação real usando os dados disponíveis.

Algoritmo de aprendizado

É o **conjunto de instruções ou passos que usamos para ensinar o modelo**.

Exemplo: um algoritmo de boosting ajusta automaticamente as variáveis e combina várias árvores de decisão para melhorar a previsão da cura de TB.

Por que queremos avaliar o modelo?

Para estimar o desempenho preditivo em dados futuros

Para aumentar o desempenho ajustando os parâmetros e selecionando o melhor modelo

Para identificar o algoritmo mais adequado para o problema em questão, ou seja, queremos comparar diferentes modelos com diferentes algoritmos

Fluxo da construção do modelo de Machine Learning

Dados

↓

Pré-processamento

↓

Divisão (Treino / Teste)

↓

Escolha do(s) algoritmo(s)

↓

Treinamento

↓

Otimização de hiperparâmetros

↓

Avaliação no teste

↓

Performance do modelo

Banco de dados

Em caso de amostragem, assume-se que os dados são representativos da população real sobre a qual o desfecho será predito.

Divisão entre treino e teste

Conjunto de Treino

- Usado para **ajustar o modelo**
- O algoritmo aprende os padrões, relações e parâmetros a partir desses dados

Conjunto de Teste

- Usado **após o treino**
- Serve para avaliar o desempenho do modelo em **dados nunca vistos**
- Evita o problema de **overfitting**

Divisão entre treino e teste

1) **Divisão aleatória**

- Os dados são embaralhados e separados aleatoriamente

Proporções mais comuns de divisão

Não existe uma regra fixa, mas as divisões mais usadas são:

70% (treino) e 30% (teste)

Para bases maiores pode-se usar 80% e 20% ou 90% e 10%

2) Para **dados coletados ao longo do tempo ->** exemplo: casos de TB de 2015 a 2024.

Pode-se separar os últimos anos para o conjunto teste e os demais para o conjunto treino

Viés pessimista

Pequenos conjuntos de dados de teste aumentam o viés pessimista.

Maior chance de “azar estatístico”

Com poucos exemplos no teste um pequeno número de previsões erradas pode derrubar bastante a métrica

Pequenas variações → grandes impactos.

Escolha do algoritmo

Exemplos:

Regressão Logística

Random Forest

LightGBM

Catboost

Xgboost

TabPFN

Treinamento do modelo e otimização de hiperparâmetros

O modelo aprende os padrões usando o **conjunto de treino**

Otimização de hiperparâmetros - normalmente feita com validação cruzada

Objetivo: melhorar a performance sem overfitting

Técnicas comuns:

- Grid Search
- Random Search
- Hyperopt

Avaliação com o conjunto de teste: performance do modelo

Comparação com a realidade

O modelo faz previsões em dados **nunca vistos**

Avaliação usando métricas adequadas:

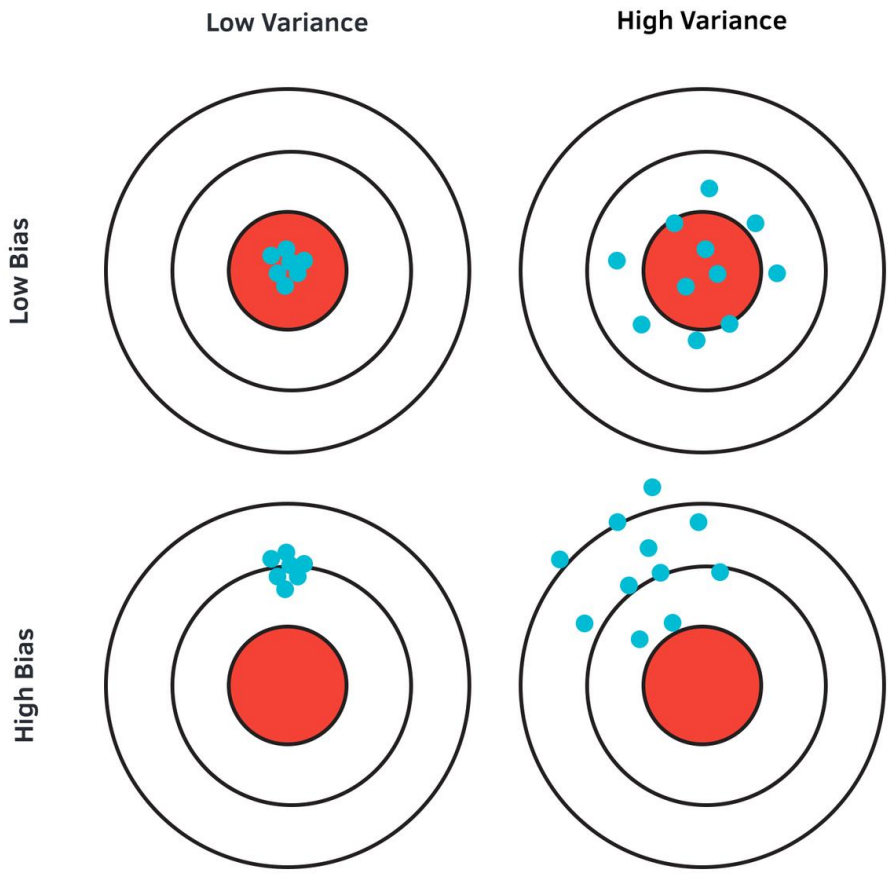
- Para modelos de classificação:
- ROC-AUC, Recall (Sens), Esp, Precision (VPP) e VPN

O conjunto de teste deve ser usado **apenas uma vez**

Assume-se a mesma performance que a dos dados futuros.

Variância e viés

Trade-off - Viés e Variância



Bias-Variance Tradeoff

Variância do modelo

O que aumenta a variância do modelo de machine learning?

- Poucos dados
- Modelo muito complexo – muitas variáveis e hiperparâmetros mal ajustados

Exemplos:

Random Forest com muitas árvores muito profundas

✓ max_depth muito alto

✓ min_samples_leaf muito baixo -> quantas OBS pode conter na folha

Boosting

✓ Muitas iterações (n_estimators alto) -> número de árvores

✓ Árvores profundas (max_depth alto)

Variância do modelo

O que aumenta a variância do modelo de machine learning?

- Muitas variáveis irrelevantes - variáveis sem relação com o alvo aumentam a chance de o modelo aprender ruído
- Falta de técnicas que “penalizam” modelos complexos

Exemplos de regularização:

reg_alpha → L1 (Lasso) → zera coeficientes irrelevantes

reg_lambda → L2 (Ridge) → reduz magnitude dos coeficientes

Sem regularização:

- O modelo se ajusta demais aos dados de treino
- Maior sensibilidade a ruído

Tabela - resumo

Fator	Exemplo	Efeito
Poucos dados	Dataset pequeno	↑ Variância
Modelo complexo	RF profundo, Boosting com muitas árvores	↑ Variância
Variáveis irrelevantes	Features s/importância	↑ Variância
Sem regularização	L1/L2 ausentes	↑ Variância

Viés no modelo

O que provoca o viés no modelo de machine learning?

Poucas variáveis (features insuficientes)

- Falta de informações relevantes
- O modelo não tem dados suficientes para aprender o padrão

Hiperparâmetros muito restritivos

- Limitações que simplificam demais o modelo:
- max_depth muito baixo (profundidade das árvores)
- n_estimators muito baixo (ensemble*)

Viés no modelo

O que provoca o viés no modelo de machine learning?

Dados pouco representativos

- Dados que não refletem bem a realidade do problema
- Padrões importantes não aparecem no treino

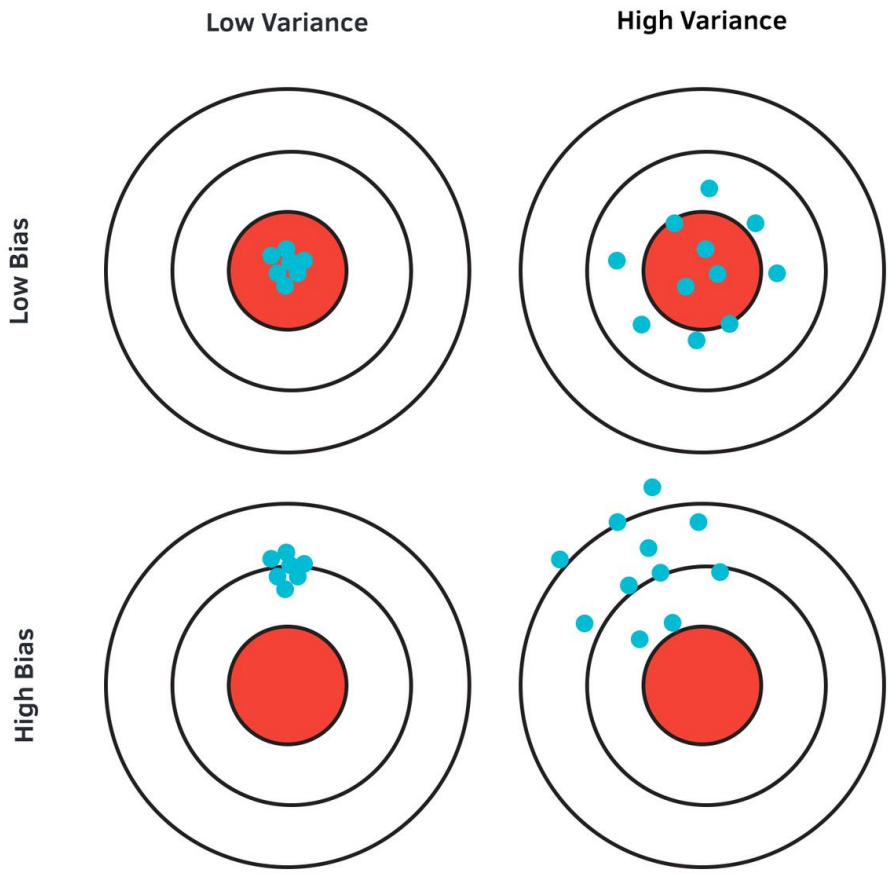
Regularização excessiva

- Penalizações muito fortes (L1 ou L2 altas)

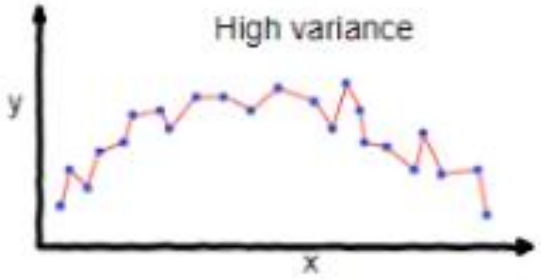
Modelo simples demais

- Baixa complexidade
- Incapaz de representar relações complexas

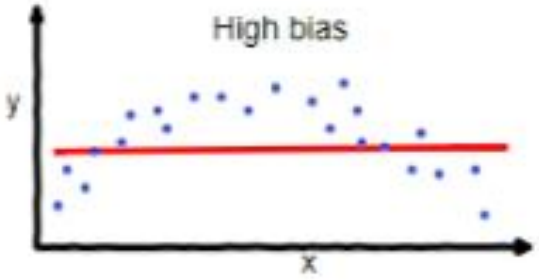
Trade-off - Viés e Variância



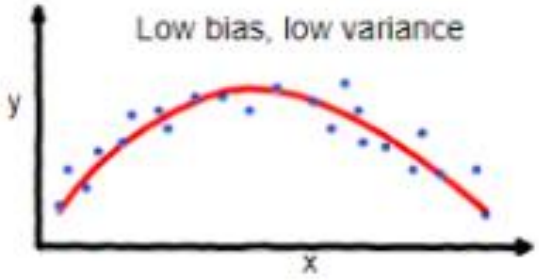
Bias-Variance Tradeoff



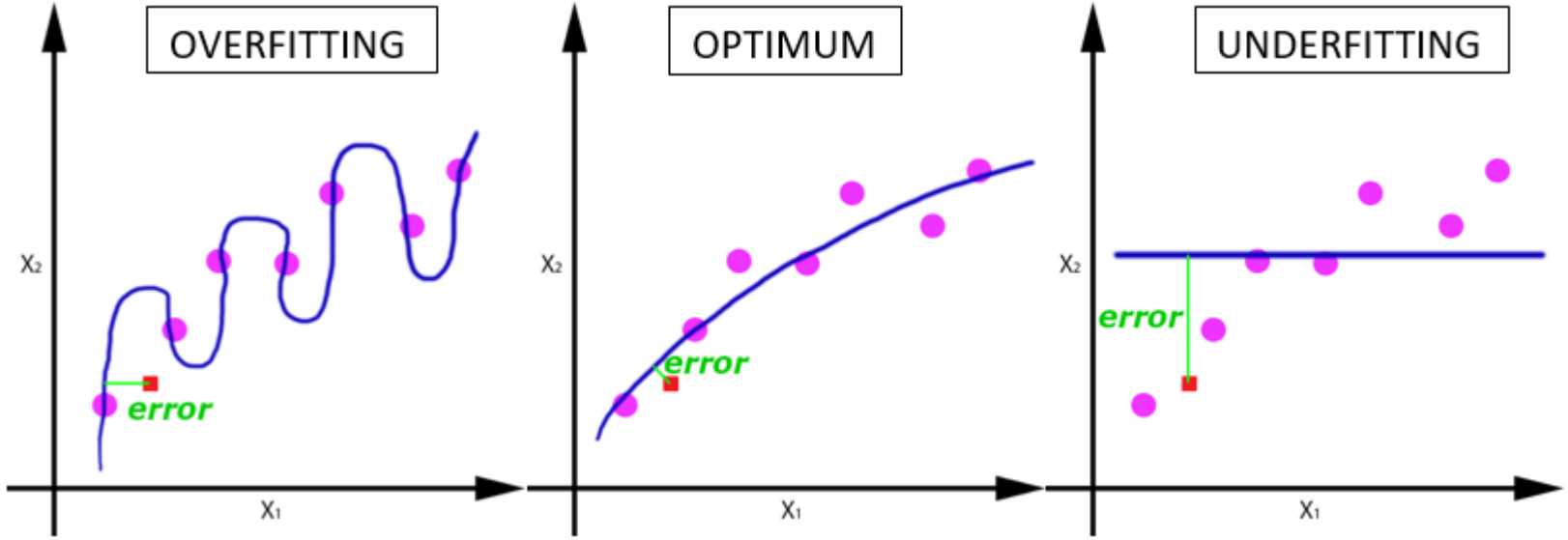
overfitting



underfitting



Good balance



Trade-off - Viés e Variância

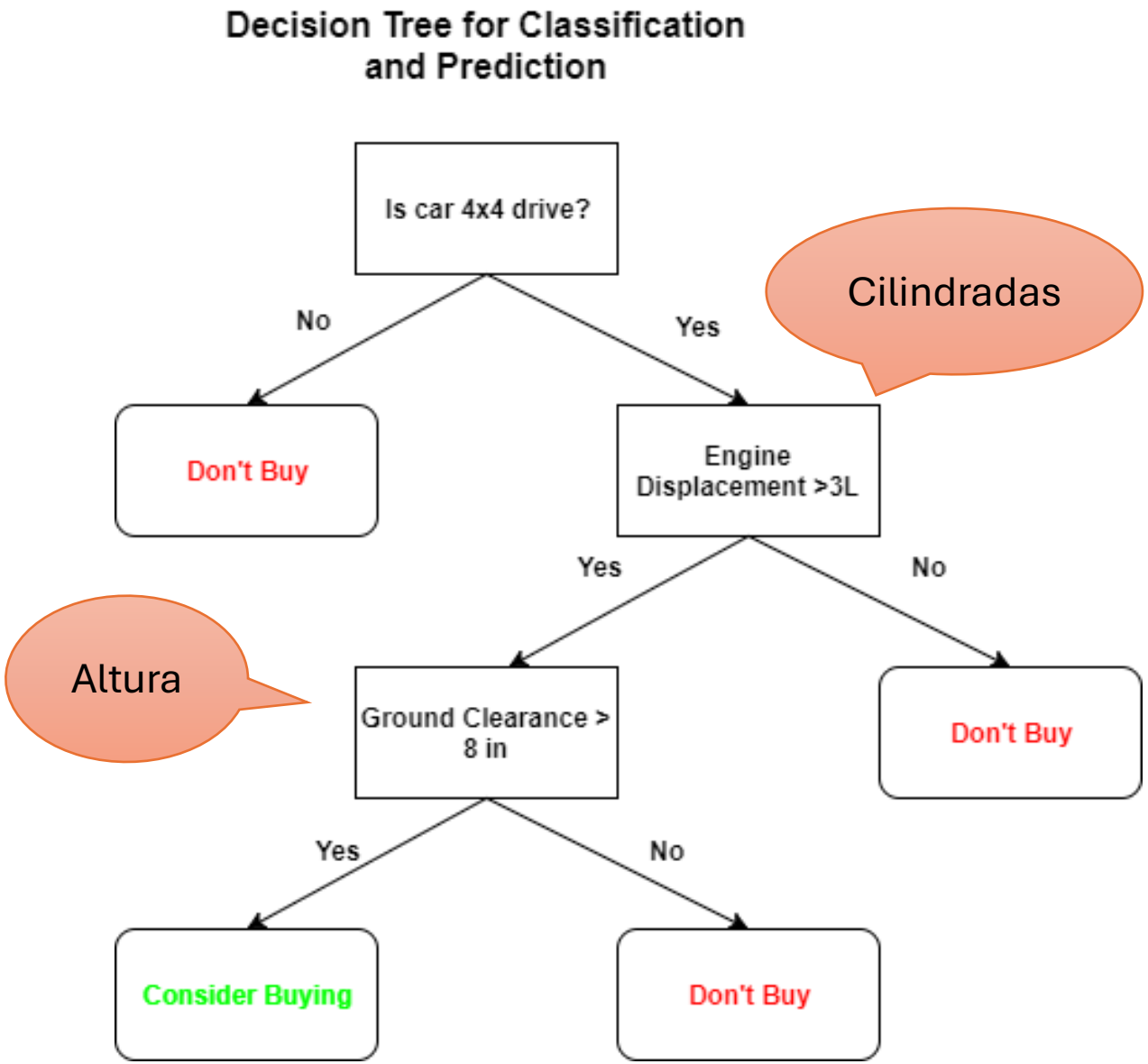
Em uma linguagem simples, podemos dizer que:

um modelo de Machine Learning com alto viés “aprendeu pouco”

e um modelo com muita variância “aprendeu demais”

Exemplos de árvores

Predição de compra ou não de carro para usar em sítio de difícil acesso



Predição da malignidade do tumor

Tamanho

- mean radius > 15?
- / \
- Não Sim
- (≤ 15) (> 15)
- / \
- texture > 20? concave points > 0.05?
- / \ / \
- Não Sim Não Sim
- Benigno Maligno Benigno Maligno

Irregularidade da borda

Otimização de hiperparâmetros

Quase todos os algoritmos têm configurações a serem especificadas

Os ajustes são feitos através dos hiperparâmetros
(em geral regularizadores – tentativas de segurar a complexidade)

Eles ajudam a otimizar o desempenho encontrando o equilíbrio entre o viés e a variância

Otimização de hiperparâmetros

Alterar valores de hiperparâmetros durante o treinamento resulta em diferentes modelos

Seleção do modelo:

Processo para encontrar o modelo com melhor desempenho, comparando o desempenho de modelos produzidos com diferentes hiperparâmetros

Otimização de hiperparâmetros

Alteramos as configurações dos hiperparâmetros para obtermos melhores desempenhos na etapa de validação.

Posteriormente avaliamos a performance final no conjunto teste.

Buscamos uma combinação ótima que maximize a performance final do modelo.

Otimização de hiperparâmetros

Feita de forma automatizada, por meio de métodos como:

- random search,
- grid search e
- otimização bayesiana (ex: Optuna, Hyperopt)

Grid Search

Passo 1 – Definição dos valores dos hiperparâmetros a serem utilizados no processo de otimização.

Passo 2 – Combina todos os valores de hiperparâmetros em uma grade de possibilidades e realiza os treinamentos com base nestas combinações

Passo 3 – Retorna a combinação de hiperparâmetros de melhor desempenho na etapa de treinamento

Exemplo Grid Search

Treinando uma **árvore de decisão** para classificar o tumor como:

- Benigno
- Maligno

Passo 1 – Definir hiperparâmetros

max_depth: [2, 3, 4]

Profundidade máxima da árvore

min_samples_split: [2, 5]

Número mínimo de exemplos p/ dividir um nó

Passo 2 – Testar combinações (Grid Search)

max_depth	min_samples_split	AUC-ROC
2	2	0.85
2	5	0.83
3	2	0.89
3	5	0.88
4	2	**0.92**
4	5	0.90

Passo 3 – Melhor combinação

Melhor resultado:

max_depth = 4

min_samples_split = 2

AUC-ROC = 0.92

Random Search

Método iterativo e automatizado de otimização que testa aleatoriamente as combinações entre os valores dos hiperparâmetros, de modo a obter uma combinação ótima para determinado número de iterações

Iterações => **define antecipadamente quantas combinações aleatórias** o algoritmo vai testar

Random Search

Passo 1 – Definição dos valores dos hiperparâmetros a serem utilizados no processo de otimização

Passo 2 – Definição do número de iterações a serem feitas

Passo 3 – Realiza sorteios nos valores de cada hiperparâmetros e cria combinações para treinamento dos dados para cada iteração

Passo 4 – Retorna a combinação de hiperparâmetros de melhor desempenho na etapa de treinamento

Exemplo Random Search

Treinando uma **árvore de decisão** para classificar o tumor como:

- Benigno
- Maligno

Passo 1 – Definir hiperparâmetros

max_depth: entre 2 e 10

min_samples_split: entre 2 e 10

Passo 2 – Definir número de iterações

5 iterações

Profundidade máxima da árvore

Número mínimo de exemplos p/ dividir um nó

Testar **5 combinações aleatórias**, não todas!

Passo 3 – Sorteio e treinamento

A cada iteração, o algoritmo sorteia valores e treina o modelo.

Iteração	max_depth	min_samples_split	AUC-ROC
1	3	7	0.86
2	9	2	**0.93**
3	5	6	0.88
4	2	9	0.82
5	7	3	0.91

Passo 4 – Melhor combinação

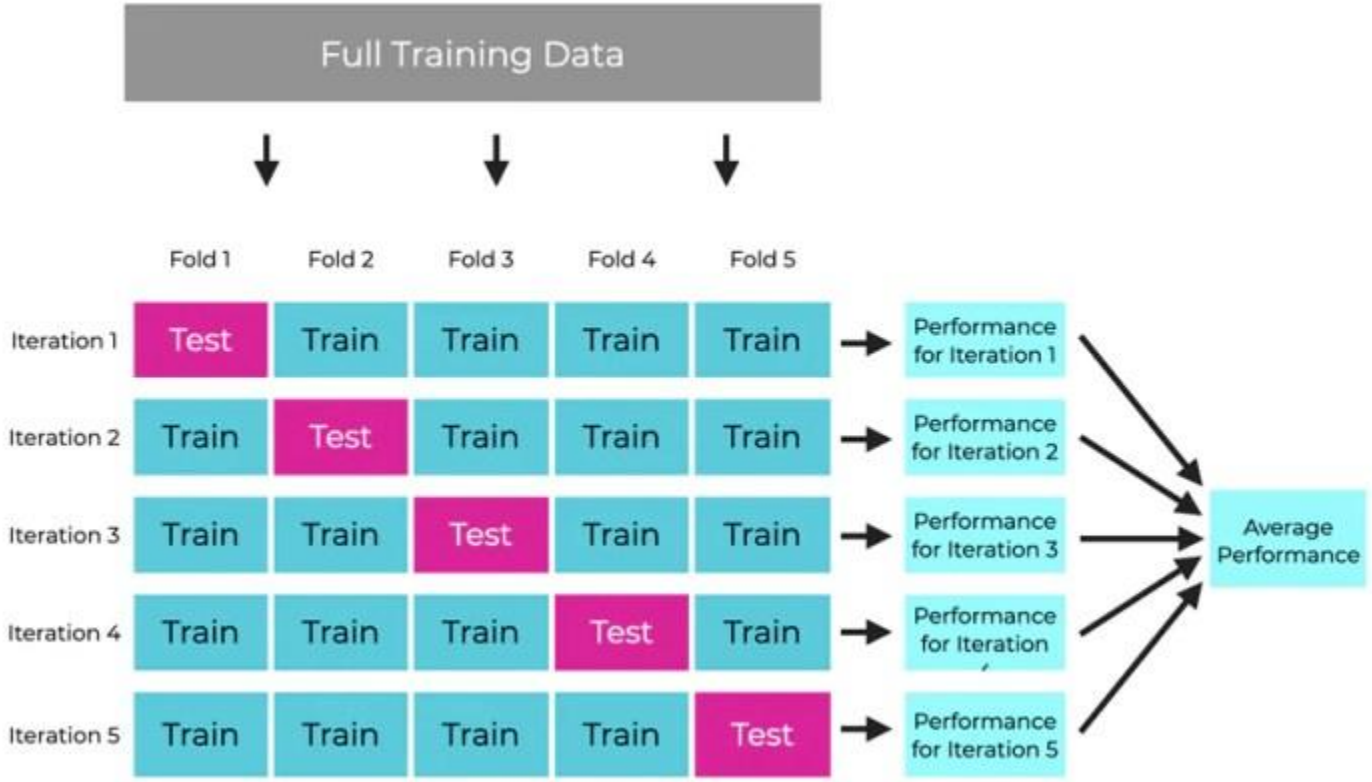
Melhor resultado:

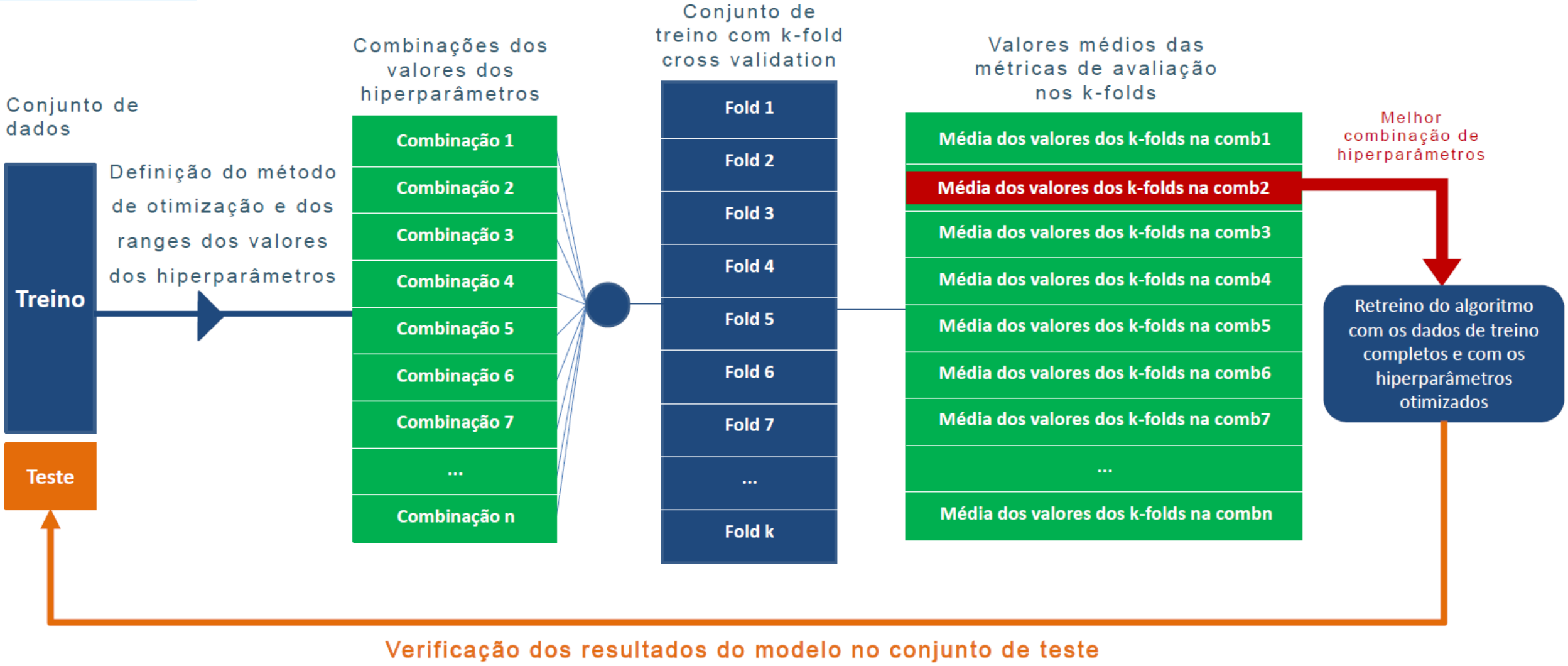
max_depth = 9

min_samples_split = 2

AUC-ROC = 0.93

Validação cruzada



Otimização de Hiperparâmetros

Funcionamento Geral